

1. What is Django?

Answer:

- Django is a high-level Python web framework.
 - It helps developers build secure and scalable web applications quickly.
 - It follows the **MVT (Model-View-Template)** architecture.
-

2. What are the features of Django?

Answer:

- Fast Development
 - Secure
 - Scalable
 - Built-in Admin Panel
 - ORM (Object Relational Mapper)
 - Authentication System
 - URL Routing
-

3. What is MVT Architecture?

Answer:

Model

- Handles database.
- Stores data.

View

- Contains business logic.
- Processes requests.

Template

- Displays HTML pages to users.

Flow:

User



URL



View



Model



Template



Response

4. Difference between Django and Flask?

Django

Flask

Full Framework Lightweight Framework

Built-in Admin No Admin

Built-in ORM External ORM

Large Projects Small Projects

5. What is ORM?

Answer:

ORM stands for **Object Relational Mapper**.

It allows us to interact with the database using Python code instead of SQL.

Example:

```
Student.objects.all()
```

instead of

```
SELECT * FROM student;
```

6. Why ORM is used?

- No SQL writing
 - Easy to understand
 - Database independent
 - Secure against SQL Injection
-

7. What is a Model?

Answer:

A Model is a Python class that represents a database table.

Example:

```
class Student(models.Model):  
    name = models.CharField(max_length=100)
```

8. What is Migration?

Answer:

Migration creates or updates database tables according to models.

Commands:

```
python manage.py makemigrations  
python manage.py migrate
```

9. Difference between makemigrations and migrate?

makemigrations

- Creates migration files.

migrate

- Applies changes to database.
-

10. What is manage.py?

Answer:

It is Django's command-line utility.

Examples:

```
python manage.py runserver  
  
python manage.py migrate
```

11. What is settings.py?

Contains project settings like:

- Installed apps
 - Database
 - Templates
 - Static files
 - Middleware
-

12. What is urls.py?

It maps URLs to Views.

Example:

```
path("home/", views.home)
```

13. What is View?

A View handles user requests and returns responses.

Example:

```
def home(request):  
    return HttpResponse("Hello")
```

14. What is Template?

Templates are HTML files used to display data.

Example:

```
<h1>{{ name }}</h1>
```

15. What is Context?

Context sends data from View to Template.

Example:

```
return render(request, "home.html", {"name": "ZYZ"})
```

16. What is render()?

Used to return an HTML page.

Example:

```
return render(request, "home.html")
```

17. What is HttpResponse?

Returns plain text.

Example:

```
return HttpResponse("Welcome")
```

18. Difference between render() and HttpResponse()

render()	HttpResponse()
----------	----------------

Returns HTML	Returns text
--------------	--------------

Uses Template	No Template
---------------	-------------

19. What is Admin Panel?

A built-in interface for managing database records.

Command:

```
python manage.py createsuperuser
```

20. What is Superuser?

An admin user with full permissions.

21. What is App in Django?

A module that performs a specific task.

Example:

- Blog
 - Student
 - Employee
-

22. Difference between Project and App?

Project	App
---------	-----

Complete Website	Small Module
------------------	--------------

Can have many apps	Belongs to one project
--------------------	------------------------

23. What is Static File?

Static files are files whose content does not change dynamically. They are used to design and style a website.

Static files include:

- CSS
 - JavaScript
 - Images
-

24. What is Media File?

Files uploaded by users.

Examples:

- Images
 - PDFs
 - Videos
-

25. What is QuerySet?

A collection of database objects.

Example:

`Student.objects.all()`

26. Difference between `get()` and `filter()`?

`get()`

- Returns one object.
- Raises an error if not found.

`filter()`

- Returns multiple objects.
 - Returns an empty QuerySet if no match.
-

27. What is Primary Key?

A unique identifier for each row.

Django automatically creates:

`id`

28. What is Foreign Key?

Used to create a relationship between two tables.

Example:

```
class Student(models.Model):  
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
```

30. What is Django Template Language (DTL)?

A language used inside HTML templates.

Example:

```
{{ name }}
```



```
{% for student in students %}  
{% endfor %}
```

31. What is CSRF?

CSRF stands for **Cross-Site Request Forgery**.

It protects forms from unauthorized requests.

```
{% csrf_token %}
```

32. What is Middleware?

Middleware processes requests and responses before they reach the view.

Examples:

- Authentication
 - Security
 - Sessions
-

33. What is Session?

Stores user data on the server between requests.

34. What is Cookie?

Stores small amounts of data in the user's browser.

35. Difference between Session and Cookie?

Session	Cookie
Stored on Server	Stored in Browser
More Secure	Less Secure

36. What is Authentication?

Verifies the identity of a user (login).

37. What is Authorization?

Determines what a logged-in user is allowed to do.

38. What is the Superuser Command?

Command

```
python manage.py createsuperuser
```

What does it do?

- Creates an admin user (superuser).
- A superuser has full permissions to manage the Django application.
- After creating it, you can log in to the Django Admin Panel.

Example

Username: admin

Email: admin@gmail.com

Password: *****

39. How to create a Django project?

```
django-admin startproject myproject
```

40. How to create an app?

```
python manage.py startapp student
```

41. How to run the server?

```
python manage.py runserver
```

42. How to create migrations?

`python manage.py makemigrations`

43. How to apply migrations?

`python manage.py migrate`

44. What is MTV vs MVC?

Django (MVT) MVC Equivalent

Model	Model
View	Controller
Template	View

45. What is `INSTALLED_APPS`?

A list in `settings.py` containing all apps used in the project.

Example:

```
INSTALLED_APPS = [  
    'student',  
]
```

46. What is `__init__.py`?

- Makes a directory a Python package.
 - Allows Python to import modules from that folder.
-

47. What is `apps.py`?

- Stores application configuration.
 - Created automatically when an app is created.
-

48. What is `models.py`?

- Used to define database tables using Python classes.
-

49. What is views.py?

- Contains functions or classes that process requests and return responses.
-

50. What is forms.py?

- Used to create and validate HTML forms.